

Data Structures — Introduction & Overview

Conceptual background (not exam-coded) — arrays, queues, linked lists, and trees, alongside the stack you already know.

CONTENTS

1. What is a Data Structure?
2. Elementary Data Representation
3. Arrays / Linear Lists
4. Stacks (Quick Recap)
5. Queues
6. Linked Lists
7. Trees
8. Operations on Data Structures
9. Comparing All Data Structures

01

What is a Data Structure?

A **data structure** is a way of **organizing and storing data** so it can be used efficiently. Just like a cupboard organizes clothes into shelves and drawers, a data structure organizes data into a particular shape or arrangement.

Different data structures suit different needs — some are great for quick access, some for maintaining strict order, and some for representing relationships between items.



Fig 1.1 — Different data structures for different jobs

Simple analogy: Choosing a data structure is like choosing a container — a bottle, a box, or a folder all "store things", but each is better suited to a different purpose.

02

Elementary Data Representation

Before organizing data into structures, it helps to know the basic ways data can be represented in memory:

Representation	Meaning
Primitive data	Single, basic values — like <code>int</code> , <code>float</code> , <code>char</code> , <code>bool</code>
Non-primitive data	Built from primitive types, can hold multiple values — arrays, lists, stacks, trees etc.
Linear representation	Elements arranged in a straight sequence, one after another
Non-linear representation	Elements connected in a branching or networked way, not a simple line

Simple rule: If you can lay the data out in a single row, it's **linear** (array, stack, queue, linked list). If it branches out, it's **non-linear** (tree, graph).

03

Arrays / Linear Lists

An **array** (or linear list) stores elements **one after another** in continuous memory, each identified by an **index**. In Python, we usually use a **list** for this purpose.

10	20	30	40
[0]	[1]	[2]	[3]

Fig 3.1 — Elements sit in fixed, indexed positions, side by side

Feature	Detail
Access	Very fast — jump directly to any index
Insertion/Deletion (middle)	Slow — other elements must shift
Order	Fixed sequence, elements keep their position

04

Stacks (Quick Recap)

You've already studied this one in depth. A quick reminder for how it fits into the bigger picture:

- Linear data structure
- Follows **LIFO** — Last In, First Out
- Insertion/deletion only from the **top**
- Implemented in Python using a **list** with `.append()` and `.pop()`

See your earlier "Stack (Data Structure)" notes for the full detail — Push, Pop, LIFO, and the complete menu-driven program.

Queues

A **queue** is a linear data structure where insertion happens at one end (the **rear**) and deletion happens at the other end (the **front**).

Think of a **ticket counter line** — the first person to join the line is the first person served.

FRONT



A leaves first (from the front)

Fig 5.1 — A queue: enters from rear, exits from front

Queue follows FIFO

FIFO — First In, First Out. The opposite of a stack's LIFO rule.

	Stack	Queue
Rule	LIFO	FIFO
Insertion	Top	Rear
Deletion	Top	Front
Example	Pile of plates	Ticket counter line

Note: Queues are not part of your current CBSE Python syllabus for implementation, but knowing FIFO vs LIFO is a common conceptual/theory question.

Linked Lists

A **linked list** is a linear data structure where each element (called a **node**) holds its data **plus a link (pointer)** to the next node. Unlike arrays, elements are **not** stored in continuous memory — they're scattered, connected only by these links.

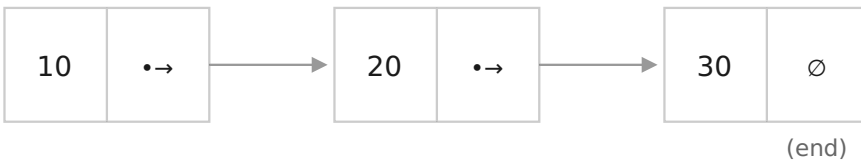


Fig 6.1 — Each node stores data + a link to the next node

Feature	Detail
Access	Slow — must follow links from the start
Insertion/Deletion	Fast — just change a few links, no shifting
Memory	Not continuous, scattered with links between nodes

Array vs linked list, in short: arrays are fast to *access* but slow to *insert/delete* in the middle; linked lists are the opposite.

07

Trees

A **tree** is a **non-linear** data structure that represents data in a **hierarchy** — like a family tree or folders on your computer, with items branching out from a starting point.

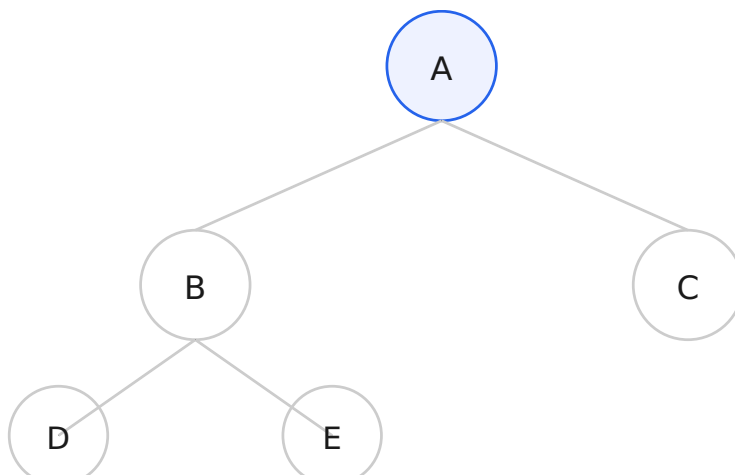


Fig 7.1 — A tree: A is the "root", B/C are its "children", D/E are B's children

Key terms

Term	Meaning
Root	The topmost node (starting point) — "A" above
Parent / Child	A node directly connected above/below another
Leaf	A node with no children (e.g. D, E, C above)
Edge	The connecting link between two nodes

Real-world examples: a computer's folder structure, an organization's reporting chart, or a family tree — all branch out from one root.

08

Operations on Data Structures

Regardless of which data structure you use, most support a similar set of basic operations:

Operation	Meaning
Traversing	Visiting/accessing every element, usually one by one
Insertion	Adding a new element
Deletion	Removing an existing element
Searching	Finding whether/where an element exists
Sorting	Arranging elements in a defined order
Merging	Combining two data structures into one

Exam tip: When asked to "define traversing/searching/sorting" for any data structure, these general definitions apply — just relate them to the specific structure being asked about.

09

Comparing All Data Structures

Structure	Type	Rule / Idea	Fast access?	Fast insert/delete?
Array / List	Linear	Indexed positions	Yes	No (middle)
Stack	Linear	LIFO	Only top	Yes (at top)
Queue	Linear	FIFO	Only front/rear	Yes (at ends)
Linked List	Linear	Nodes with links	No	Yes
Tree	Non-linear	Hierarchy (root→leaves)	Depends	Depends

Covered in this notebook: what a data structure is, elementary data representation, a conceptual look at arrays, a stack recap, queues (FIFO), linked lists, trees, common operations, and a comparison of all five. This is background knowledge — for CBSE Python coding, only the **Stack** is required.